AMIGA GURU MEDITATION ERROR

A5K

A Theoretical Design Proposition for an Amiga 5000-Like System

The machine Commodore never shipped — an Amiga 5000-like design, specified from the Copper up.

Guru Meditation #0000000A.00005000

7.0714 MHz	**4K**	**×4**	**≈$280**
COPPER TICK	SCANOUT CEILING	PCIE LANES	PROTOTYPE BOM

ABSTRACT

The Amiga’s custom chips were never a graphics card bolted to a CPU. They were a set of narrow, fixed-function DMA engines sharing one memory, driven by a beam-synchronous instruction stream. This document argues that the same *method* — not the same silicon — is what a modern machine should re-adopt, and specifies one: an FPGA card with 1 GB of DDR3L Chip RAM (2 GB upgradeable) presenting an Agnus-class DMA scheduler, a planar Denise-class scanout, a minterm blitter and an MC-1 Copper over PCIe, targeting a RISC-V host and an open-source toolchain. It also answers the obvious objection — why not simply use a GPU — and is explicit about where this machine loses. Three appendices turn the argument into engineering: the Copper instruction set and register map, a Migen implementation skeleton, and a bill of materials with indicative 2026 pricing.

STARTED	30 October 2026
REVISION	Rev. AP — 3 November 2026 — the BOM states that alternates with different pinouts are board revisions, not part swaps
STATUS	Design study; no hardware built
REFERENCE CONCEPT	“A5K”
CONTENTS	9 sections · 3 appendices · 6 tables · 1 figure

Amiga and Commodore are trademarks of their respective owners. This is an independent design study, not affiliated with, endorsed by, or sponsored by any rights holder. No logo or registered-mark symbol is used. The design is provided as is, without warranty of any kind, and is not for safety-critical, medical, life-support, aviation or aerospace use. See LEGAL.md for the full notice.

Contents

1. What “the latest model of that era” actually was 1

2. The design ethos, distilled 1

3. Chip-by-chip: the modern re-issue 3

4. Getting it onto a modern bus 4

5. Which host processor? 5

6. A concrete reference build 5

7. Software 7

8. What you’d lose 7

9. Why not just use a GPU? 7

Appendix A — The Copper, specified **10**

Appendix B — Implementation skeleton **13**

Appendix C — Bill of materials **15**

Appendix D — Where this is most likely wrong **17**

Tables and figures

Table 1 — Chip-by-chip: the modern re-issue 3

Table 2 — Against a modern GPU: the honest scoreboard 9

Table 3 — MC-1 opcode map 10

Table 4 — Register aperture and key registers 11

Table 5 — Plane attributes, and what they may not do 13

Table 6 — Bill of materials, reference build 15

Figure 1 — The card-as-chipset shape 4

Page numbers refer to the folios printed at the foot of each page. The cover is unnumbered; the contents page is *ii*.

> **Question:** *What would a new and modern version of the old Commodore Amiga (but the latest model of that era, or of that time period) look like today? With all custom chip subsystems following the same design ethos and methodology, what would be the most appropriate modern versions of those same kinds of chips be today, and how would we get to use it on a modern bus and modern form factor — whether PCIe or something else? And could it run on a RISC-V CPU, or what would be best, and all free and open source, if possible?*

Answer: Great question — and a fun one, because the Amiga is one of the few old machines whose architecture maps *upward* into modern silicon almost 1:1. Let me anchor it first, then build the modern version.

1. What “the latest model of that era” actually was

The last real Amiga big box was the **A4000T** (1994–97, 68040/68060, Zorro III, 16-bit Paula, 2 MB Chip / up to 128 MB Fast). But the *actual* “next Amiga” was cancelled silicon:

- **AAA** (Advanced Amiga Architecture, ~1993–94): 32-bit chipset, 16-bit chunky *and* planar, 8-channel 16-bit audio, 64-bit blitter, a much wider Copper, 8 bitplanes, 1280×1024. Killed in 1994.
- **Hombre** (1995): PA-RISC CPU + a custom 3D chipset, essentially an Amiga-derived games console. Killed too.

So a “new modern Amiga” = **AAA’s successor, with Hombre’s 3D ambitions**, and it should keep the *method*, not the mask layout.

What it is not. A5K is not a clone. It contains no Commodore code, ships no Kickstart ROM, and offers no binary compatibility with Amiga software. It is an independent re-issue of the architecture’s ideas — which, unlike its code, its ROMs and its marks, are free to use: the relevant patents expired by 2012, and the semiconductor mask-work term is ten years.

2. The design ethos, distilled

Strip the nostalgia and the Amiga chipset is really six engineering rules:

1. **Fixed-function coprocessors**, each with a narrow, dumb job — not one general-purpose engine.
2. **Everyone is a DMA master** into one shared, arbitrated memory (“Chip RAM”).
3. **Raster/beam-synchronous programming** — the Copper, not vblank-locked APIs.
4. **Memory-mapped registers, no privileged API.** The hardware *is* the API; poking it is legal.
5. **Zero-copy**: display fetches the same bytes the CPU wrote.
6. **Deterministic cycle timing** — demos depend on counting DMA slots, so timing is a contract, not best-effort.

Everything below is judged against those six.

3. Chip-by-chip: the modern re-issue

Table 1. Each 1985 block and its present-day descendant.

Amiga chip	Job	Modern equivalent (production silicon)	Open-source IP to build it from
Agnus	DMA controller, beam counter, memory arbiter, blitter/copper/audio/disk DMA	Display-fetch scheduler + QoS memory crossbar (what every SoC display controller's fetch unit is)	`LiteDRAM`, LiteX Wishbone/AXI crossbar + a custom slot-table arbiter
Denise	Planar scanout, sprites, priority, HAM/EHB, genlock	Display engine / DPU with **overlay planes** and hardware cursor — the sprite successor (Appendix A.5)	`LiteVideo`, `hdl-util/hdmi`, custom planar fetch pipeline
Blitter	Minterm ALU on 3 sources, line draw, area fill	2D engine: STM32 **DMA2D/Chrom-ART**, NXP **PXP/ePDC**, GPU 2D ring	Custom (it's small), or `Nyuzi`/`Vortex` if you want compute
Copper	Programmable display-list processor (wait/skip/move)	**GPU command processor** (AMD CP, NVIDIA front-end), DRM/KMS atomic commit, console "display lists" (PS2 VU1, N64 RSP)	Custom microsequencer (trivial in an FPGA); specified in Appendix A
Paula	4-ch variable-rate 8-bit DMA audio, floppy, serial, POT	Audio DMA engine + I2S/TDM codec; SDIO/eMMC; UART	`LiteSDCard`, `LiteEth`, custom audio DMA
CIA 8520	Timers, GPIO, interrupts	Always-on / housekeeping MCU block	`VexRiscv` / `NEORV32`
Gary/Buster	Bus glue, address decode	NoC / interconnect	LiteX, AXI4, Wishbone

SIDEBAR

Why Paula bundled audio and disk — and why a PC doesn't

Paula (8364) carried four audio channels, the floppy controller, the serial port, the paddle inputs and the interrupt controller — shipped in 1985, still in the A4000T in 1997. AGA replaced Agnus and Denise and left Paula alone.

The PC went the other way: a sound card became a codec, then a stack of kernel driver, userspace mixer and a shared 48 kHz clock every application resamples to. The card is a peripheral on a bus; the OS owns the clock, and the program waits.

Paula is a *memory client*. Each channel is a DMA stream with its own period register, summed in hardware, reading the same Chip RAM as the display. Latency is one sample period; no mixer, no resampler, no daemon. The price was 8-bit samples and four voices — right in 1985, wrong by 1992.

The rebuild keeps the model and fixes the arithmetic: one oscillator for audio and video, so A/V sync is a ratio not a resample; audio DMA in the display's slot table, so worst-case latency is known before shipping; I^{2}S to an eight-channel 24-bit DAC; storage still raw-bitstream, with software deciding what the words mean.

For 24/192 playback with DSP, a PC wins and §9 concedes it. For audio that must land *with* something, a scheduler in the path is the defect.

The important realization: **the modern equivalents already exist and are still fixed-function.** Overlay planes *are* sprites and playfields with priority. A display engine's fetch/QoS arbiter *is* Agnus stealing cycles. A GPU's command processor *is* a Copper. The industry didn't abandon the Amiga model — it renamed it and hid it behind a driver.

What’s genuinely gone: the *planar* bitplane format, HAM/EHB, and publicly documented registers.

4. Getting it onto a modern bus

Three viable homes, in increasing order of “real Amiga-ness”. In every case the shape is the same — a host on one side of a fast link, the chipset on the other:

For the avoidance of doubt: the reference product is an add-in card that requires an x86-64 host PC. It has no CPU, no boot firmware and no operating system of its own, and it cannot be fitted to a Commodore Amiga — those machines have Zorro II and Zorro III, not PCIe. Minimum host requirements, the reasoning behind the ×4 slot, and the module and mainboard alternatives are set out in *A5K — Technical Specifications*.

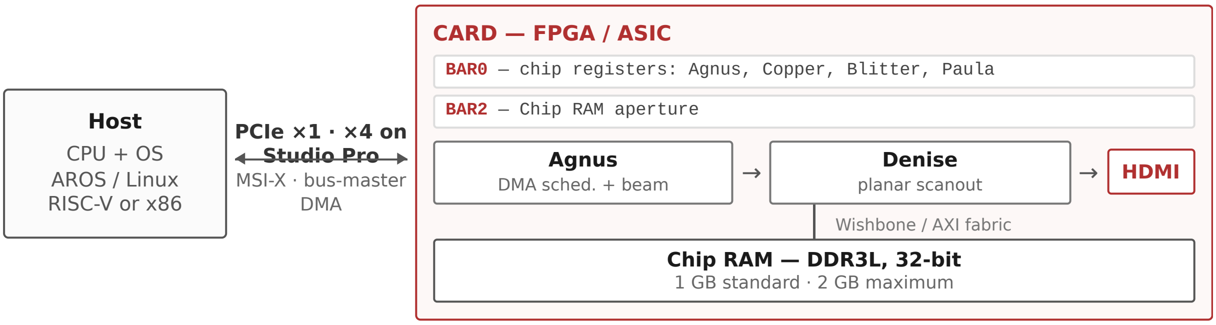
Host
CPU + OS
AROS / Linux
RISC-V or x86
PCIe ×1 · ×4 on Studio Pro
MSI-X · bus-master DMA
CARD — FPGA / ASIC
BAR0 – chip registers: Agnus, Copper, Blitter, Paula
BAR2 – Chip RAM aperture
Agnus
DMA sched. + beam
Denise
planar scanout
HDMI
Wishbone / AXI fabric
Chip RAM — DDR3L, 32-bit
1 GB standard · 2 GB maximum

Figure 1. The card-as-chipset shape shared by all three options: one register aperture, one memory aperture, one command list, one display pipe.

A. PCIe add-in card — “the Amiga as a GPU.” This is the natural fit for that shape. Every modern GPU is already: PCIe endpoint + local VRAM + display engine + command processor + DMA. Make that card *behave like a chipset*:

Key point: **PCIe latency (~1 µs) doesn’t hurt, because the Amiga model was designed to be latency-tolerant.** All raster-critical work lives in the Copper list *on the card*; the CPU only sets up lists ahead of time. That’s exactly why the architecture scales to a 4K display engine with a 7.0714 MHz Copper tick.

B. CXL Type-2 accelerator. If you want the literal “Chip RAM” semantics — the chipset DMAs coherently into host memory, CPU and blitter see one address space — CXL.mem is the modern Agnus bus. More exotic, but it’s the honest translation.

C. Chiplet / SoC tile. Put the whole chipset as an IP block on an SoC interconnect (AXI/ACE/CHI) next to a RISC-V cluster. **Microchip PolarFire SoC** (RISC-V + FPGA fabric) is the closest off-the-shelf “new Amiga motherboard” today; you drop the chipset into the fabric and the CPU cluster talks AXI to it.

Form factors: PCIe low-profile card (desktop), **M.2 2280** (laptop/SFF — a 4-lane card fits, and the display goes out over the card’s own HDMI or via eDP), USB4/Thunderbolt external box (eGPU style), or a mini-ITX/ATX board with the SoC + fabric on-board for the “big box” feel (and Zorro-style expansion via PCIe slots).

5. Which host processor?

The card has no application CPU. Its only processor is a housekeeping core — a VexRiscv at 50–150 MHz that runs timers, GPIO, the SD card and interrupt aggregation. It has no MMU, no filesystem and no application loader. Everything a user sees runs on the host, on the other side of PCIe.

So the choice below is a choice of *host*, and it changes nothing inside the card:

- **Native 68k (soft core).** Open cores exist: **fx68k** (cycle-accurate 68000, used in MiSTer) and **TG68K**. This is the only path to *binary* Amiga compatibility on real hardware — but it caps you around 68000/010 performance and big-endian is a feature, not a bug.
- **RISC-V (RV64GC).** Best open, best ecosystem, best tooling (VexRiscv, CVA6, BOOM, Rocket, XiangShan). Downside: little-endian by default, so planar bitmaps and 68k structs need byteswapping — do it in the blitter/scanout and it's free. Run UAE/AROS for compatibility.
- **OpenPOWER.** Big-endian, open ISA (Microwatt, Libre-SOC), which is *spiritually* the most Amiga-correct choice (68k is big-endian; planar graphics assume it). Practically, weaker ecosystem.

What the numbers look like. On the FPGA itself, a VexRiscv synthesises at roughly 50 MHz on an ECP5-85 and 100–150 MHz on an Artix-7 200T; a PolarFire SoC offers four hardened RV64 cores at 667 MHz. None of this is the point. The on-card core supervises timers, GPIO and interrupts — the pixels come from the Copper, the blitter and the planes. The heavy work belongs on the host, which may be anything from a 2 GHz RISC-V board to a 256-core x86 machine. Full figures are in *A5K — Technical Specifications*.

> **RECOMMENDATION**
>
> **Reference host today: x86-64.** Every desktop has one, Linux support is mature, and the first driver will be written there — which is why *A5K — Technical Specifications* lists it as a requirement.
>
> **Destination: RISC-V.** Open, well-tooled, and the natural processor for a standalone A5K. It needs a driver written for it and, for a self-contained machine, an AROS port that does not yet exist.
>
> **Purist: OpenPOWER** — big-endian and open, which suits planar graphics, at the cost of a thinner ecosystem.
>
> The 68k core is for running old binaries, not for the machine's main work.

6. A concrete reference build

Call it the **A5K**:

- **Carrier:** Xilinx Artix-7 100T — four GTP transceivers, which is what 4K60 output requires: one lane for PCIe, three for HDMI. The Lattice ECP5-85 is only viable for a 1080p variant, because its 3.2 Gb/s SerDes cannot carry 4K60 TMDS. (both have usable open flows via **Yosys + nextpnr**; larger designs may need Vivado for the FPGA but keep the *design* in Migen/LiteX so it's portable).

- **SoC glue: LiteX** — gives you LitePCIe, LiteDRAM, LiteSDCard, LiteEth, Wishbone/AXI bridges, and a VexRiscv if you want the whole machine on one board.
- **Chipset IP:** Agnus-equivalent (slot-table DMA arbiter + beam counter), Denise-equivalent (planar scanout + planes + priority + HAM), Blitter (minterm ALU + Bresenham), Copper (microsequencer), Paula (audio DMA + SDIO + UART), CIA (VexRiscv).
- **Video out: 4K60 SDR HDMI as primary**, straight from fabric; **1080p60 DisplayPort as secondary**, through a bridge. Both are driven at once; sensing decides which display supplies the EDID. HDMI is sensed first, DisplayPort second, with DVI, S-Video and composite reserved for later boards. **HDR10 is a Studio feature** — 10-bit 4:2:2 over HDMI, 10-bit 4:4:4 over DisplayPort; 4K60 4:4:4 on HDMI requires HDMI 2.1 and is not implemented on any model. On the Studio models both connectors run 4K60 HDR at once, but only one of them can be composited graphics: the second is video, a crop of the same frame, or an independent 1080p beam. Two independent 4K HDR graphics rasters need the 512-bit Pro II. C+ has no capture, and is mirrored. Refresh is a **fixed set** up to 1080p240, **never variable**. See *A5K — The Family*, §5.3.
- **Audio: eight channels, 24-bit** — four 3.5 mm stereo jacks, TOSLINK, and 8-channel LPCM over HDMI and DisplayPort. **A standard 10-pin front-panel header** carries headphones and a microphone, with its own stereo DAC so the rear eight channels are never broken. **Stereo is the factory default on every output** — 2-channel LPCM on HDMI and DisplayPort, 2-channel PCM on TOSLINK, a downmix on jack 1 — and the user's chosen mode becomes the power-on default from then on. TOSLINK carries two channels or a compressed bitstream, never eight channels of PCM. Two audio clock families, with the 44.1 family as the default. See *A5K — The Family*, §5.5.
- **RAM: 1 GB DDR3L as standard** — two 4 Gb devices on a 32-bit bus — with **2 GB as a drop-in upgrade** using 8 Gb parts in the same footprint. The 32-bit pointer path caps Chip RAM at **4 GiB**, of which about 3.5 GB remains usable once registers and apertures are reserved; beyond that, the 64-bit variant widens both the bus and the address path. Host memory is reachable over PCIe for "Fast RAM" and is not limited by the card at all.
- **Display model:** keep planar + chunky, keep HAM/EHB, keep planes (Appendix A.5), keep priority. Add overlay planes so a modern windowing system can sit on top without giving up the old modes.
- **Sync primitive:** expose a **beam-synchronous fence** — "wait until scanline N then fire this Copper list" — so host software gets raster sync back (`drm_vblank`/`drm_syncobj` semantics map exactly).

The parallelism budget. About twenty fixed-function units run at once — the Copper, the blitter, eight planes and up to eight audio streams — because each does one thing forever at a fixed rate, not because anything is scheduled. The bound is memory, not logic: at 4K60, eight planar bitplanes cost 0.5 GB/s, a single full-screen ARGB plane costs 2.0 GB/s, and the 32-bit DDR3L bus supplies 6.4 GB/s. Adding CPU cores does not add throughput; it adds contention on the same memory.

How that budget is actually enforced — the slot table, its two currencies of 141.4 ns ticks and 6.7 ns slots, and the rules a Copper list must respect — is specified separately in *A5K — The Slot Table*.

7. Software

- **OS: AROS** is the open-source answer — it's a source-compatible reimplementation of the AmigaOS 3.x API, already running on x86/ARM/68k/PPC. A port to RISC-V plus a `chipset.hidd` driver for the PCIe device is the intended route to a real Amiga-like OS on modern silicon. *No such port exists for this hardware today; it is a target, not a result.* (AmigaOS 4 and MorphOS are closed/PPC; ApolloOS is an AROS fork for Vampire.)
- **Alternatively, Linux + DRM:** the mapping is almost comically direct — **atomic commit = Copper list**, **plane properties = playfield/sprite registers**, **vblank event = beam interrupt**, **syncobj = Copper WAIT**. Then run host-ed AROS or UAE on top.
- **Firmware/openness:** everything above (LiteX, Yosys, nextpnr, AROS, VexRiscv, fx68k) is free/open. The only closed piece is typically the FPGA vendor's place-and-route for big parts and the HDMI PHY.

8. What you'd lose

Be clear-eyed: you'd lose **cycle-exact OCS/ECS/AGA behaviour** unless the Copper and Agnus are literally bit-identical (emulators already do this better than new silicon will). You'd lose the mask-ROM-era "one chip, one job, documented register" simplicity the moment you add memory protection. And planar/HAM are dead ends for 3D — Hombre's 3D would have to be a *second* fixed-function pipeline, or a small shader core bolted alongside, which starts to violate rule #1.

But the *ethos* survives: a machine where the CPU sets up command lists and specialized silicon does the pixel work in lockstep with the beam, on one shared memory, with the registers exposed. That's a modern Amiga — and honestly, it's a better description of a modern GPU than most people realize.

9. Why not just use a GPU?

This is the first question a hardware reviewer asks, and it deserves a straight answer rather than a defence. Every modern SoC already contains a massively parallel rasteriser with programmable shaders, and Vulkan and Metal expose it with far less overhead than DirectX or OpenGL ever did — you can bypass both of those APIs today and talk almost directly to the hardware. So why build a chipset at all?

Because the chipset is not in the pixel business. It is in the **timing** business. The two machines optimise different quantities, and the difference is measurable rather than sentimental.

9.1 What a GPU is for

A GPU is a throughput machine: thousands of lanes, deep queues, and a scheduler whose only job is to keep them busy. Its contract is *eventual* completion — a frame arrives, eventually, in an order the driver and the scheduler chose. That is the correct contract for a 3D scene, and it is why a mid-range card beats anything described here by three orders of magnitude at shading, texturing and compute.

Nothing in this document competes with that. If the workload is ten million triangles with physically based shading, the answer is a GPU, and this chipset has nothing to contribute.

9.2 What a GPU cannot promise

- **Bounded latency.** A frame's completion time depends on queue depth, scheduler policy, power state and what else is resident. No register tells you that line 214 will be fetched at dot 40; the Copper's entire value is that you can know that in advance.
- **A stable timeline.** Vulkan gives you fences and semaphores — ordering guarantees, not timing guarantees. `vkQueueSubmit` returns immediately; the work happens when it happens.
- **Ownership of the display.** The OS compositor owns scanout. An application cannot change a register between two scanlines, because it cannot see the scanlines.
- **Direct access.** Poking a register has no equivalent: there is a validation layer, a command buffer, a kernel-mode scheduler and usually a firmware blob between the program and the raster.
- **One address space.** A GPU has VRAM, a driver-managed heap and a copy engine. "The display fetches the bytes the CPU wrote" is a property of Chip RAM, not of a PCIe BAR.

None of this is a criticism of GPUs. It is the difference between a *scheduler* and a *sequencer*. Both execute instructions; only one of them promises to execute the next one 141 ns from now.

9.3 What shaders are good for — and where they belong

Shaders are the right tool for per-pixel work, and this design does not forbid them; it declines to put them in the scanout path. A programmable pixel stage destroys the property that makes the rest of the machine useful: you can no longer count cycles, because the cost of a pixel now depends on what it computes and what data it waits for.

The honest architecture is therefore a hybrid, described in §9.6: let a GPU do the shading, and let the chipset do what it is built for — composition, scanout and timing.

9.4 Where this machine loses, plainly

It is not a competitor to a current PC and should never be sold as one. On the axes that most people measure, it loses outright:

- **3D and compute.** No triangle rate worth quoting, no texture units, no shader cores, no ray tracing. A 2026 integrated GPU is faster than this entire card.
- **Memory bandwidth.** A 128-bit DDR3 path delivers tens of gigabytes per second; a modern discrete card delivers hundreds, and the slot table deliberately gives much of ours to display fetches.

- **Tooling and ecosystem.** Nothing comparable to SPIR-V, no profilers, no vendors, and an operating system (AROS) with a tiny fraction of the software.
- **Price.** At $280 in parts it is not even cheap next to a used office PC with a capable integrated GPU.

9.5 The honest scoreboard

Table 2. Where each machine wins, stated without flattery.

Axis	Modern PC + GPU	A5K card	Wins
Shading throughput	Billions of triangles/s, fully programmable	None in the scanout path	PC
Compute	Vulkan/Metal compute, tensor units	None (an optional GPU tile would supply it)	PC
Worst-case frame latency	Unbounded: scheduler, compositor, driver	One scanline, by construction	A5K
Sub-frame control	None — the app cannot see scanlines	Per-scanline register writes from the Copper	A5K
Code between app and pixel	10^6–10^7 lines of driver and firmware	~10^4 lines of RTL you can read	A5K
Display power	5–30 W for the GPU alone	1–2 W for the whole chipset	A5K
Software catalogue	Effectively everything	AROS plus 68k legacy	PC
Price / performance	Unbeatable, and improving yearly	Not a meaningful metric	PC

9.6 You can have both

Nothing in the design prevents a GPU from being one of the plane sources. The pattern is the one every modern display controller already uses: the GPU renders into a Chip RAM buffer, the buffer is exposed as a plane, and the Copper keeps ownership of the cursor, the OSD and the raster splits around it. The GPU's latency then stops mattering, because scanout never waits for it — you double-buffer, and if a frame is late the previous one is shown again, exactly as a compositor does.

That hybrid is how a shipping machine would actually be built: GPU for shading, chipset for composition, timing and I/O. What it cannot do is give you both properties in the same pixel — a pixel is either produced by a scheduler or by a sequencer.

9.7 So what is it for?

Not a PC replacement. These are the places where the architecture is not merely quaint but genuinely better:

- **Live video and genlock.** Where the frame must land on house sync, and a compositor's jitter *is* the defect.
- **Instrumentation and test equipment.** A display whose timing is a documented contract rather than an emergent property.
- **Hardware-accurate retro machines.** A real chipset rather than an emulator with a shim.
- **Teaching.** A complete graphics stack — RTL, driver, OS — that one person can read end to end.
- **Art and research.** Raster-level control with modern I/O and no vendor in the path.

A GPU is the right answer to “how do I draw this fast?”. The Amiga chipset is the right answer to “when exactly will this pixel appear?”. They are different questions, and a serious machine should be able to answer both — which is precisely why §9.6 exists.

Appendix A — The Copper, specified

A design like this only becomes real when the Copper is nailed down to bits. What follows is **MC-1**: a self-consistent re-issue that keeps the 1985 programming model — a list of instructions executed in lockstep with the video beam, with no interrupts and no CPU involvement — but widens it to 64-bit instructions, a 16-bit register index and a 24-bit beam address. It is deliberately *not* binary-compatible with OCS/AGA lists; the compatibility front-end at the end of this appendix restores that for legacy software.

A.1 Instruction format

Every instruction is one 64-bit word, aligned to 8 bytes. The list is an array of these words in Chip RAM; the Copper fetches and retires one per tick.

Bits	Field	Meaning
63–60	OP	Opcode (8 defined of 16)
59–56	FLAGS	Execution qualifiers (see below)
55–32	A	Operand A: register index, beam position, or list offset
31–0	B	Operand B: data word, or mask

Flag	Name	Effect
0	WAIT_BLIT	Hold until the blitter is idle before retiring the instruction
1	WAIT_SLOT	Hold until the requested DMA slot has been fetched (cycle-exact sync)
2	IRQ_ON_DONE	Raise the Copper interrupt when this instruction retires
3	LINE_SYNC	Align to the start of the next scanline rather than the current one

A.2 Opcodes

Table 3. The eight defined MC-1 opcodes.

Op	Mnemonic	Operation	Typical use
0x0	MOVE	`reg[A[15:0]] ← B`	Set any chip register: colours, pointers, control
0x1	WAIT	Stall until `(beam & B) == (A & B)`	“Wait for line N, then…” — the raster split
0x2	SKIP	If the same condition holds, skip the next instruction	Conditional colour changes without branching
0x3	JUMP	`PC ← list_base + 8·A`	Loops inside one list
0x4	CALL	Push PC, then JUMP	Shared sub-lists (a “Copper library”)
0x5	RET	Pop PC	Return from CALL
0x6	END	Reload from COP2LC if armed; raise VBlank IRQ	End of frame; double-buffered lists
0x7	IRQ	Raise interrupt `A[7:0]` , payload B	Host notification mid-frame (rare, expensive)

Beam packing. For WAIT and SKIP, operand A holds a packed position and B a mask over the same fields: `A = (V << 12) | H`. V is the line number (0–4095; 4K uses 2,250). H is the horizontal position **in ticks, not dots** — 105 positions per line at 1080p, 53 at 4K, 453 at 640×512 — so twelve bits cover every mode, and the field does not widen when a faster tick domain arrives. Counting H in dots would need thirteen bits at 4K, and the beam cannot be addressed more finely than one instruction anyway. Because the mask is applied to both sides, `0x0FFF000` matches the vertical field alone: the classic "wait for line N" idiom.

Timing. The Copper retires one instruction per tick, and the tick is **7.0714 MHz** — 141.4 ns, exactly 21 slots of the 148.5 MHz bus. That integer ratio is deliberate: 7.09 does not divide 148.5, and an approximation drifts five slots per line, which would make the claim that timing is countable false. The tick is 0.26% slower than the Amiga's, and exact. A MOVE to a display register costs one tick; a satisfied WAIT costs one tick and falls through; **a fetch costs one slot**, because an MC-1 instruction is 64 bits and a slot is 64 bits. For the rest of the frame the Copper is idle — which is why it can afford to be dumb. At 4K a line holds 53 positions instead of 105, but the tick stays 141.4 ns.

```
; --- classic raster split, MC-1 assembler syntax ---------------------
        MOVE   0x0340, 0x00F0        ; COLOR00 := Amiga blue
        WAIT   0x064000, 0x0FFF000   ; hold until the beam reaches line 100
        MOVE   0x0340, 0x0F00        ; COLOR00 := white
        MOVE   0x0344, 0x0F0F        ; COLOR01 := pale grey
        WAIT   0x0B4000, 0x0FFF000   ; hold until line 180
        MOVE   0x0340, 0x00F0        ; back to blue
        END                          ; re-arm from COP2LC on the next frame
```

Listing 1. Nine words of Copper produce a gradient bar that no CPU has to touch after setup.

A.3 Register map

The register file is a flat 4 KiB window in BAR0 (extended to 64 KiB in the ASIC revision), accessed as 32-bit words with 16-bit addressing. Reads of write-only registers return the last value written, so a driver can always save and restore state.

Table 4. Aperture map, then the registers a programmer actually touches.

Range	Block
0x0000–0x00FF	Copper, beam counter and list control
0x0100–0x01FF	Denise: bitplanes, display window, video modes
0x0200–0x02FF	Planes (legacy sprites): position, size, priority, merge order; collision detect
0x0300–0x03FF	Palette — 256 entries, 32-bit RGBA (classic 0x0180 lives here)
0x0400–0x04FF	Blitter
0x0500–0x05FF	Paula: audio, disk, serial, POT
0x0600–0x06FF	DMA and interrupt control
0x0700–0x07FF	CIA: timers, GPIO, time-of-day
0x0800–0x08FF	Genlock and video timing
0x0900–0x0FFF	Reserved for the next revision (3D, expansion)

Register	Addr	Width	Function
COP1LC	0x0000	32	Primary Copper list base (8-byte aligned, Chip RAM)
COP2LC	0x0004	32	List reloaded by END — the double-buffer hook
COPJMP1/2	0x0008	8	Strobe: restart from COP1LC / COP2LC immediately
COPCON	0x000C	8	bit 0 legacy decode, bit 1 Copper IRQ enable
VPOSR	0x0010	16	Current line number

Register	Addr	Width	Function
VHPOSR	0x0014	32	Line in [23:12], tick position in [11:0] — the same units a WAIT compares
BEAMCON0	0x0018	16	Tick divisor [3:1] (reads 0 — fixed 7.0714 MHz), line length, interlace, sync polarity
HPOSR	0x001C	16	Fine horizontal position in dot clocks, read-only, for sub-tick measurement
DMACON	0x0600	16	Per-channel DMA enable (bitplane, blitter, copper, audio, disk)
INTENA / INTREQ	0x0604 / 0x0608	16	Interrupt enable mask / request-and-clear
BPLCON0–3	0x0100	16	Depth, HAM, EHB, dual playfield, resolution
BPL1PT–BPL8PT	0x0110+	32	Bitplane base pointers
DIWSTRT / DIWSTOP	0x0130	16	Display window start / stop
DDFSTRT / DDFSTOP	0x0134	16	Data fetch window (the shifter’s horizontal trim)
BLTCON0/1	0x0400	16	Minterm, shift, fill mode, line mode, blitter-busy flag
BLTSIZE	0x040C	16	Width × height — writing it starts the blit
AUDxLC / AUDxPER	0x0500+	32 / 16	Sample pointer / period (sample rate), per channel
DSKLEN	0x0520	16	Disk DMA length; write-twice arms the transfer
CIA PRA/PRB/TA/TB	0x0700+	8	Timers, GPIO and TOD, 68k-compatible shapes

A.4 Legacy compatibility

Setting `COPCON.LEGACY` switches the front-end into the 1985 encoding: pairs of 16-bit words, MOVE when bit 0 of the first word is clear, WAIT/SKIP with the vertical field in bits 2–8 and the horizontal in the second word. The translator runs at list-load time, costs one extra tick per two words, and preserves the original’s illegal-opcode behaviour (an invalid instruction halts the Copper and sets the Copper-bad bit), because demos and copy-protection both rely on that. Compatibility is therefore a property of the list, not of the machine: one card can run an AGA demo and a 4K AROS desktop in adjacent windows, each with its own Copper dialect.

> **DESIGN NOTE**
>
> The temptation is to “improve” the Copper with loops, conditions and a stack — which is what modern GPU command processors did, and why they now need a compiler. The list above adds only JUMP/CALL/RET, and nothing with a data-dependent branch, because the Copper’s value is that its execution time is *knowable* before it runs. Every instruction added for convenience is a cycle the demo programmer can no longer count.

A.5 Planes, not sprites

The word *sprite* survives in this document as a compatibility alias, not as a specification. What OCS and AGA called a sprite was a strictly budgeted object: eight of them, 16 pixels wide (64 on AGA), three colours plus transparency, fetched by DMA in planar form, with a collision-detection unit bolted alongside. None of that is worth reproducing in 2026. All of it is worth *emulating*, because existing software expects it.

What the design specifies instead is a set of **planes** — the block every modern display controller already contains, documented here rather than hidden behind a driver.

Table 5. The plane block, and the constraints that keep it Amiga-like.

Attribute	Specification
Count	8 hardware planes, plus 8 composited in the merge stage
Format	ARGB8888 linear; 1–8 bitplane planar in legacy mode
Size	Up to 4096 × 4096 pixels
Position	Integer x/y per plane; no sub-pixel placement — scanout is integral
Blending	None, constant alpha, or premultiplied alpha, selected per plane
Priority	4-bit level; the original resolution order is preserved in legacy mode
Source	Any Chip RAM address, fetched per line, never cached
Update	Any attribute writable by the Copper mid-frame; pointers double-buffered
Collision	`CLXDAT` computed in the merge stage for legacy lists; disabled otherwise
Cost	One slot per line per plane in the slot table — never free, always countable

Four rules keep the block from drifting into a GPU:

1. A plane is **fetched**, not composited in software.
2. Its position is a **register**, so the Copper can move it between scanlines.
3. It occupies a **known slice of the line budget**, counted by the slot table before the frame begins.
4. It cannot sample a texture, run a shader, or blend in a non-linear colour space. If you need that, it is not a plane — it is a GPU, and §9 sets out what you give up.

The last rule is the important one, and it is a design constraint rather than a shortcoming. An overlay plane that can do everything *is* a GPU; a GPU has no timing contract; and the timing contract is the only thing this machine has that a PC does not.

Appendix B — Implementation skeleton

Two files are enough to see the shape: the Copper itself, and the top level that hangs the whole chipset off a LiteX SoC. Both are sketches — the real blocks are a few hundred lines each, and the hard parts (arbitration, clock-domain crossing, the blitter’s slot stealing) are exactly the parts elided. They are written in Migen because that is what LiteX is built on, and because a hardware description that fits on a page is the best argument that the design is still simple.

B.1 The Copper

```
# copper.py — MC-1 display-list processor (sketch; imports elided)
from migen import *

MOVE, WAIT, SKIP, JUMP, CALL, RET, END, IRQ = range(8)

class Copper(Module):
    """Retires one 64-bit instruction per 7.0714 MHz tick."""

    def __init__(self, beam, reg_wr, mem):
        self.list_base = Signal(32)      # COP1LC / COP2LC, 8-byte aligned
        self.enable    = Signal()
        self.jump      = Signal()        # COPJMP1/2 strobe
        self.stall     = Signal()        # chip-bus slot not granted
        self.irq_ev    = Signal()
        self.irq_num   = Signal(8)

        pc    = Signal(32)
        ins   = Signal(64)
        ready = Signal()

        # ---- fetch: one word per granted slot -------------------------
        self.comb += [
            mem.adr.eq(pc),
            mem.stb.eq(self.enable & ~self.stall & ~ready),
            mem.we.eq(0),
        ]
        self.sync += [
            If(mem.ack, ins.eq(mem.dat_r), ready.eq(1)),
            If(~mem.stb, ready.eq(0)),
        ]

        # ---- execute --------------------------------------------------
        cond = (beam & ins[31:0]) == (ins[55:32] & ins[31:0])
        self.sync += [
            If(self.jump | ~self.enable,
                pc.eq(self.list_base), ready.eq(0),
            ).Elif(ready & ~self.stall,
                Case(ins[63:60], {
                    MOVE: [reg_wr.adr.eq(ins[39:32]),
                           reg_wr.dat.eq(ins[31:0]),
                           reg_wr.we.eq(1), pc.eq(pc + 8)],
                    WAIT: [If(cond, pc.eq(pc + 8))],         # else hold
                    SKIP: [If(cond, pc.eq(pc + 16))
                            .Else(pc.eq(pc + 8))],
                    JUMP: [pc.eq(self.list_base + (ins[55:32] & 0xFFFFF) * 8)],
                    CALL: [pc.eq(self.list_base + (ins[55:32] & 0xFFFFF) * 8)],
                    RET:  [pc.eq(pc + 8)],                   # stack omitted
                    END:  [pc.eq(self.list_base), self.irq_ev.eq(1)],
                    IRQ:  [self.irq_num.eq(ins[55:48]),
                           self.irq_ev.eq(1), pc.eq(pc + 8)],
                })
            )
        ]
```

Listing 2. Instruction fetch, decode and retire; the return stack, blitter-slot handshake and register-file decode are omitted.

B.2 Wiring it into a LiteX SoC

Hanging the blocks off a bus is where the design stops being a sketch. Before the code, the honest summary of the effort:

> **Where the difficulty actually lives.** Not in the Copper — it is a 300-line state machine. The hard parts are (1) the *slot table*, a fixed schedule of which master gets Chip RAM on which cycle, because that schedule *is* the machine's timing contract; (2) clock-domain crossing between the 7.0714 MHz chip domain, the 148.5 MHz slot domain and the 594 MHz pixel domain; and (3) making the PCIe BAR2 aperture coherent with the on-card DDR controller so a host write is visible to the next scanline fetch without a flush.

```
# soc.py — chipset as an AXI/Wishbone peripheral (sketch)
from litex.soc.integration.soc_core import SoCCore
from litex.soc.integration.builder import Builder
from litedram.core import LiteDRAMCore
from litepcie.core import LitePCIeEndpoint

from copper import Copper
from agnus import Agnus
from denise import Denise
from blitter import Blitter

class AmigaSoC(SoCCore):
    def __init__(self, platform):
        SoCCore.__init__(self, platform, clk_freq=100e6,
                         cpu_type="vexriscv",
                         integrated_rom_size=0x8000)

        # one DDR3 port, many masters: Agnus owns the arbiter
        self.submodules.dram  = LiteDRAMCore(platform.request("ddr3"))
        self.submodules.agnus = Agnus(self.dram.crossbar.get_port())
        self.submodules.denise  = Denise(timing=platform.request("hdmi"))
        self.submodules.blitter = Blitter(self.agnus.slot)
        self.submodules.copper  = Copper(self.agnus.beam,
                                         self.agnus.reg_wr,
                                         self.agnus.port_a)

        # host view: BAR0 = registers, BAR2 = chip RAM window
        self.submodules.pcie = LitePCIeEndpoint(
            platform.request("pcie_x4"), max_pending_requests=8)
        self.bus.add_slave("chipset", self.agnus.bus,
                           region=0x0000_0000, size=0x0000_1000)

        # 1080p60 out of a 7.0714 MHz tick: the beam counter scales,
        # the Copper does not
        self.sync += self.agnus.line_len.eq(2200)
        self.comb += self.denise.pix_clk.eq(self.crg.clk148_5)

def main():
    from litex.build.generic_platform import *
    from amiga_platform import Platform      # ECP5 / Artix-7 pinout
    soc = AmigaSoC(Platform())
    Builder(soc, output_dir="build").build()
```

Listing 3. The whole machine, as a peripheral. Real code adds clock domains, the PCIe BAR decoder and the slot table.

Appendix C — Bill of materials

The reference build is **A5K (C+)**, around a Xilinx Artix-7 100T — the cheapest part with four GTP transceivers, which is what 4K60 output requires: one lane for PCIe, three for HDMI. The Studio models are costed in §C.1. Prices are indicative single-unit distributor figures for Q4 2026 in USD; FPGAs and DDR move a lot, so treat these as order-of-magnitude, not quotes.

> **Why this is dearer than an ECP5 design.** An ECP5-85 costs $89 and cannot drive 4K60 — its SerDes top out at 3.2 Gb/s, well short of HDMI 2.0's 5.94. The FPGA change alone accounts for about $80 of the difference; the DisplayPort bridge, the HDMI receiver and the extra connectors account for the rest.

Table 6. Reference build — per board, parts only.

#	Item	Example part	Qty	Unit	Ext.
1	FPGA	Xilinx XC7A100T-2FGG484 (4 GTP)	1	$170.00	$170.00
2	Configuration flash	Winbond W25Q128JVSIQ	1	$2.60	$2.60
3	DDR3L, 512 MB	Micron MT41K256M16TW-107	2	$7.40	$14.80
4	Video clock generator	Skyworks Si5351A-B-GT	1	$1.90	$1.90
5	DisplayPort bridge (1080p)	HDMI-to-DP converter	1	$3.00	$3.00
6	HDMI connector + ESD	TE 1903015-1, TI TPD4E02B04	1	$1.10	$1.10
7	DisplayPort connector + ESD	DP receptacle, ESD array	1	$1.50	$1.50
8	Audio DAC, 8-channel	AKM AK4458VN · *alt. Cirrus CS4385*	1	$7.50	$7.50
9	Audio out: optical + analogue	TOSLINK transmitter, 4 × 3.5 mm jacks	1	$8.30	$8.30
10	Audio clocks	22.5792 / 24.576 MHz	2	$0.90	$1.80
11	microSD socket	Hirose DM3AT-SF-PEJM5	1	$0.55	$0.55
12	Power: bucks + LDOs	TI TPS54620, TPS7A90 ×2	1	$6.20	$6.20
13	Passives, crystals, ESD, fan, bracket	various	1	$12.00	$12.00
14	Headphone DAC + amplifier	TI PCM5102A, TPA6130A2	1	$4.50	$4.50
15	Microphone ADC + front-panel header	AKM AK5720VT · *alt. Cirrus CS5368*, 10-pin HD-Audio	1	$3.30	$3.30

#	Item	Example part	Qty	Unit	Ext.
	Parts subtotal				**$239.05**

Build cost (5-board prototype run)	Per lot	Per board	Board + parts
PCB — 4-layer, low-profile PCIe, ENIG	$60	$12.00	$251.05
SMT assembly, two sides, 5 off	$120	$24.00	$263.05
Test, programming, reflow stencil	$25	$5.00	$268.05
Reference card, working prototype	**$230**	**$41.00**	**≈ $280**

C.1 Variants and volume

Variant	Difference	5 off	100 off
Reference — A5K (C+)	Artix-7 100T, 4 GTP, PCIe ×1, 4K60 mirrored	≈ $280	≈ $223
Studio Pro — A5K Studio Pro	Kintex-7 325T (-3), 16 GTP, two beams, PCIe ×4	≈ $1,225	≈ $960

C.2 Host and tooling

Item	Note	Indicative
RISC-V host board	Any PCIe-equipped board (VisionFive 2, Milk-V, PolarFire Icicle)	$80 – $700
FPGA toolchain	Yosys + nextpnr + openFPGALoader	$0
2D/3D IP	LiteX, LitePCIe, LiteDRAM, AROS, VexRiscv	$0
ASIC conversion (optional)	28 nm shuttle, shared mask, 100 dies	$60k – $150k

> **What these numbers exclude.** Engineering time (the dominant cost by far), enclosures, the host machine, PSU, tax and shipping, certification, and any NRE for the ASIC path. They also exclude the one thing money cannot buy quickly: a driver stack, which is months of work even with AROS already written.

C.3 Prototype path

The sensible order is software first: the Copper and slot table can be validated against existing Amiga software before anyone spins a board.

1. **Simulation.** Write the blocks in Migen, run them under Verilator and dump frames to PNG; a real A1200 supplies the reference output, register write by register write.
2. **One board, one mode.** An ECP5 board with DVI out: 640×512 planar scanout and a live Copper list, no PCIe — where the slot table is proven and most schedule bugs surface.
3. **Attach the host.** Add LitePCIe x1 Gen1 and BAR0, then write the AROS HIDD or the Linux DRM driver against the register map in Appendix A. The driver is the long pole: start it while the hardware is still an FPGA.

Alternates. The converters carry a second source first, because that is where allocation bites: AKM **AK4458** ↔ Cirrus **CS4385**, AKM **AK5720** ↔ Cirrus **CS5368**. The USB PHY and headphone amp follow suit. A different pinout means a **board revision, not a part swap**.

Only then is the BOM worth ordering — the PCB is the cheapest item and the last to exist. The honest reading is that the silicon is cheap and the software is expensive: the inversion that killed the original Amiga.

Appendix D — Where this is most likely wrong

READ THIS BEFORE TRUSTING ANY NUMBER

Nothing in this document has been built, simulated or measured. Every figure is arithmetic from datasheets, standards and clock rates. What follows is what that arithmetic assumes, ordered by what a reviewer should attack first. **This appendix is the real test of the design.** A specification that cannot name its own weak points is a brochure.

Assumption	If it is wrong
Two slot clocks — 148.5 and 148.35 MHz, tick ÷21 in both	A /1.001 rate on the wrong clock slips one frame in 1,001: a visible repeat every 16.7 s
tRFC is 350 ns	It is 260 ns for the 4 Gb parts in the bill of materials. The budget improves by 1.2 points — this document overstates its own cost
256-bit closes at 148.5 MHz on a -2	A -3 speed grade, +20–25% on the FPGA. If 512 bits also fails, there is no Pro II on this board
One Copper lane in twenty-one is enough	WAIT positions drift by a tick, and the timing contract — the whole product — is void
Buffers of one tRFC are enough	A permanent pixel-hold every 7.8 µs: the worst kind of fault, because it looks like software
The bill of materials — $280 for the reference card	±20% is normal. The wedge stops being a wedge at +25%

Settling them is mechanical: the EDID of the target displays for the first, the ordering part number for the second, place and route with negative slack reported for the third, tests 2 and 3 of the slot table for the next two, and three quotes at five and at one hundred units for the last.

The full register — one undefined item, eight unverified claims, six assumptions, and the four objections already answered — is *A5K-Frailties.md*. If you find something, quote its identifier; every revision cites the ones it answers.

LEGAL NOTICE

Not a clone. This work contains no Commodore code, ships no Kickstart ROM and offers no binary compatibility with Amiga software. It is an independent re-issue of the architecture’s ideas; the relevant patents expired by 2012 and the mask-work term is ten years.

Amiga and Commodore are trademarks of their respective owners. This is an independent design study, not affiliated with, endorsed by or sponsored by any rights holder. Provided “as is”, without warranty of any kind, and not for safety-critical, medical, life-support, aviation or aerospace use. To the maximum extent permitted by law the author accepts no liability for any loss arising from its use. **The author publishes information and does not manufacture or sell goods.** Full notice: LEGAL.md. Not legal advice.

Licensing. Hardware design: CERN-OHL-S v2. Documents: CC BY 4.0. Scripts: MIT. The complete notice is in LEGAL.md, with the licence texts in LICENSE-hardware, LICENSE-docs and LICENSE-scripts.

How to cite. Ryan David Hall (AKA "ProphetPX"). (2026). *A5K — A Theoretical Design Proposition for an Amiga 5000-Like System* (2026).